
Autonomous Systems and Mobile Robots

MiniLab 2 – Summer Term 2026

Issued: 19.06.2026 **Due:** 01.07.2026, 23:59 **Points:** 50 (pass at 25)

Professor

Prof. Dr. Malte Schilling
mschilling@uni-muenster.de

Tutor

Janosch Bajorath
j.bajorath@uni-muenster.de

In MiniLab 1 your differential-drive robot learned to perceive and react: it read its LiDAR and steered itself through an obstacle field without a map. This MiniLab gives that robot an *arm* — and with it, the ability to interact with the world.

You build up to this in two stages. In **Task 1** you *build and tune the arm control stack*: forward- and inverse-kinematics *services*, a closed-loop joint controller whose gains you set and tune, an *action server* that drives the arm through a trajectory, and a mission node that uses it to push a block. You verify the whole stack on a fixed, pedestal-mounted arm before any navigation is involved. In **Task 2** you *run the mission*: mount that same arm on your mobile robot and send it through a straight corridor. A hinged “door” blocks the far end; your robot must push it open with the arm and drive through into the room beyond. Task 1 is the toolbox; Task 2 is the job.

The two tasks map onto the Perceive–Reason–Act loop you have used since Week 1: the *control stack* reasons and acts (kinematics + closed-loop motion), and the *mission* (push_block_mission, then corridor_mission) runs the full P–R–A loop and drives the stack. The architecture section details the stack; the mission protocol is introduced there too.

Submission: Submit a single ZIP of your ROS2 workspace to LearnWeb by **01.07.2026, 23:59**. Name the file `minilab2_lastname1_lastname2.zip` (add a third name if applicable). The archive must build from scratch with `colcon build` and contain every package needed to run both tasks. Omit build artefacts (`build/`, `install/`, `log/`).

Validity rule for Task 2. Your corridor navigation and arm targeting must be driven by LiDAR sensor data, and the arm must be commanded through your `ExecuteTrajectory` action client. A submission that hard-codes obstacle positions, substitutes fixed time delays for detection, or bypasses the action client receives **0 points for Task 2**. Task 1 may use known waypoints — the fixed arm has no sensing.

The Two Link Planar Arm

Both tasks use the same manipulator: the two-link planar arm from the Week 6 notebook. Two revolute joints — shoulder and elbow — carry two links to an end-effector at the tip, both rotating about a common axis so the whole arm moves in a single plane. Unlike the notebook's fixed 1.0 m links, the provided arm is *configurable*. Its link lengths are defined once in a YAML config file — one per arm size — which is the single source of truth: the URDF builds its geometry from it and the kinematics reads the same values, so the two cannot disagree. Figure 1 shows it in Gazebo.

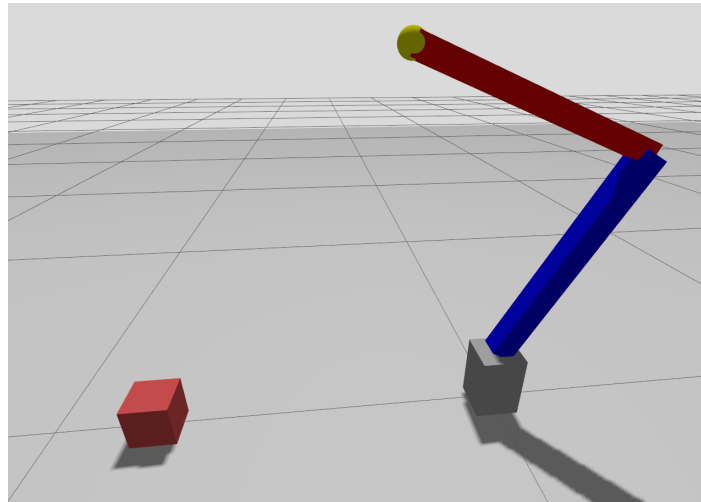


Figure 1: The provided two-link arm in Gazebo: the two links (blue = shoulder, red = elbow) reach the end-effector (yellow tool) on its base; a push block sits on the ground in front.

The arm is torque-controlled: a PID controller closes the loop on the joint *efforts*, driving each joint toward a commanded angle and holding it there against gravity — the closed-loop feedback controller from Week 1, now on a real manipulator. In Task 1 you configure and tune that controller and command the arm through it; in Task 2 you mount the same arm on your mobile robot and use it to manipulate the environment.

Architecture

You submit a single workspace. Figure 2 shows its complete layout: what is provided (including a ready-to-use reference robot you can swap for your own MiniLab 1 robot), and the files you create in each task. The tasks refer to these files by name.

```
ws/src/
|-- asmr_arm_description/           # provided
|   |-- urdf/arm.urdf.xacro        # arm (+ optional pedestal)
|   |-- urdf/arm.ros2_control.xacro # hw interface template + gz plugin
|   |-- config/arm_dimensions_pedestal.yaml # Task 1 geometry (l1=l2=0.5 m)
|   |-- config/arm_dimensions.yaml    # you create (Task 2)
|   |-- config/controllers_pedestal.yaml # template: complete + tune (Task 1)
|   |-- config/controllers_mobile.yaml # you create (Task 2)
|   '-- worlds/push_block_world.sdf   # Task 1 world
|-- asmr_arm_interfaces/           # provided (pinned contract)
|   |-- srv/ComputeFK.srv          # angles -> end-effector pos
|   |-- srv/ComputeIK.srv         # target pos -> angles
|   '-- action/ExecuteTrajectory.action # waypoints -> motion + feedback
|-- asmr_arm_bringup/             # build, Task 1: launch + bridge
|   '-- launch/push_block_world.launch.py # you write this (Task 1)
|-- asmr_arm_control/             # build, Task 1 (ament_python)
|   |-- asmr_arm_control/
|   |   |-- kinematics_server.py    # serves ComputeFK / ComputeIK
|   |   '-- trajectory_server.py    # ExecuteTrajectory server
|-- asmr_arm_mission/             # build, Task 1 (ament_python)
|   '-- asmr_arm_mission/
|   |   '-- push_block_mission.py    # home -> approach -> push
|-- my_robot_description/         # provided base (arm-less); mount arm (Task 2)
|   '-- worlds/corridor_world.sdf   # Task 2 world (poses in header)
|-- my_robot_bringup/             # provided; you write the launch (Task 2)
|   '-- launch/corridor_world.launch.py # you write this (Task 2)
|-- my_robot_control/             # provided
|-- my_robot_perception/          # provided (trimmed)
'-- my_robot_mission/             # build, Task 2: corridor mission
    '-- my_robot_mission/
        '-- corridor_mission.py      # FSM: drive + arm + enter room
```

Figure 2: The submitted workspace — every package and the key files you create, labelled by origin (provided base, or built in Task 1 / Task 2).

What you create are the four packages *not* provided or functional — `asmr_arm_control`, `asmr_arm_mission` and `asmr_arm_bringup` (Task 1) and `my_robot_mission` (Task 2); create them with `ros2 pkg create`. You also write the two scenario launch files (`push_block_world.launch.py`, `corridor_world.launch.py`), the arm's `ros_gz` bridge configuration for Task 1, and — for Task 2 — the mobile arm's `arm_dimensions.yaml` and `controllers_mobile.yaml`, modelled on the provided pedestal versions.

Once everything is running, this control stack turns a desired end-effector position into arm motion; Figure 3 shows the wiring, and Task 2 places the whole stack behind a mobile robot and a LiDAR-driven mission.

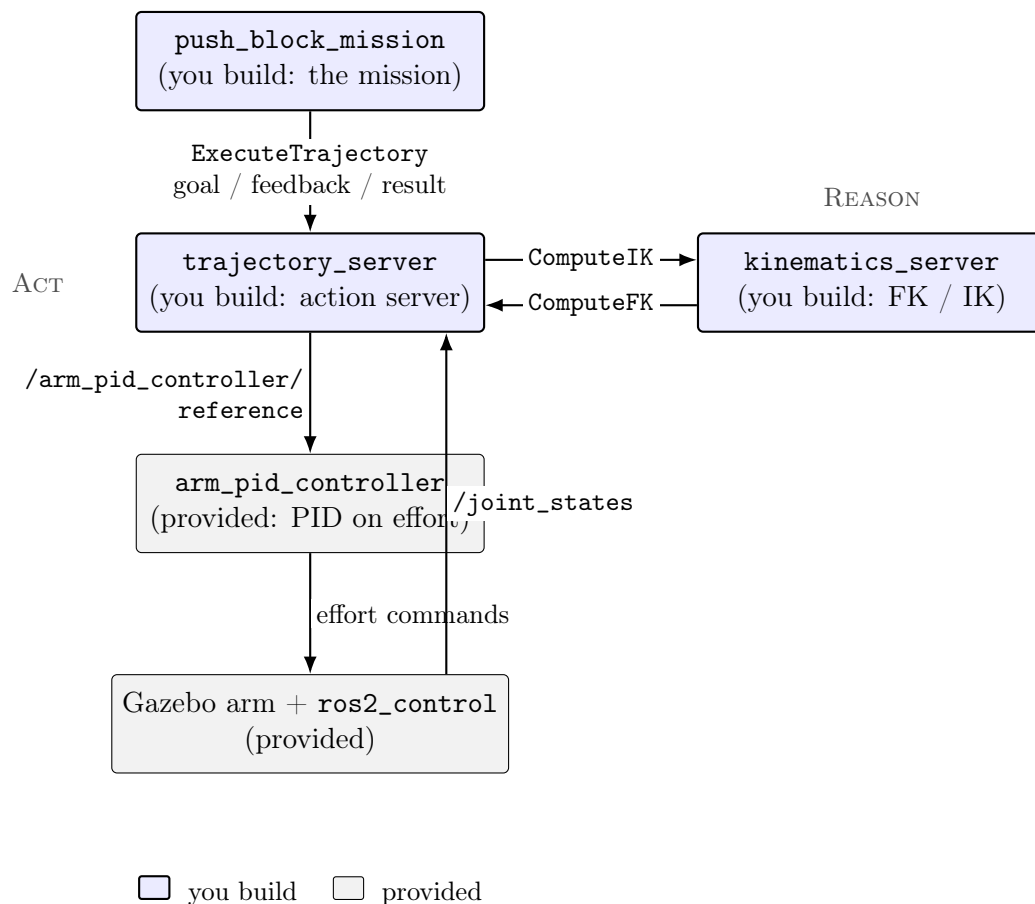


Figure 3: The Task 1 arm control stack: `push_block_mission` sends a goal, `trajectory_server` generates the reference trajectory (calling `kinematics_server`) and publishes it to `arm_pid_controller`, which closes the loop on the effort interface. `/joint_states` feeds back to `trajectory_server`. In Task 2, `corridor_mission` takes the role of `push_block_mission`.

Note the division of labour. `push_block_mission` never computes kinematics and never commands joints — it only sends an `ExecuteTrajectory` goal and reads the feedback and result. `trajectory_server` owns reference generation: it asks `kinematics_server` for joint angles, interpolates a smooth stream of setpoints, and publishes them as `control_msgs/MultiDOFCommand` to `/arm_pid_controller/reference`. A single setpoint per target would make that reference jump; the dense stream is what makes the arm sweep there smoothly — generating it is what `trajectory_server` is for. `arm_pid_controller` closes the loop: it tracks that reference against the measured joint state and drives the joints via effort. (`/joint_states`, published by the `joint_state_broadcaster`, is what `trajectory_server` reads to follow progress and settle each waypoint.) `kinematics_server` holds the FK/IK maths from Week 6. In

Task 2, `corridor_mission` takes the role of `push_block_mission`: it decides *where* the arm must reach from the LiDAR, then sends the same `ExecuteTrajectory` goal.

Mission Protocols: Behaviour-Based Control

A mission runs the full Perceive–Reason–Act loop, but *how* it decides what to do next is a design choice. The simplest mission is a fixed, pre-planned sequence of steps — enough when nothing in the world changes (Task 1’s push). A mission that must *react* to its surroundings instead composes a set of *behaviours* — each a sensor-driven action (“while this holds, do that”) — and switches between them as conditions change. This is behaviour-based control.

The usual way to organise it is a **finite state machine**: the robot is in exactly one state at a time, each state runs one behaviour, and a *transition* fires when a sensor condition is met, moving to the next state. A flat `if/elif` script with no explicit state works for trivial cases but tangles quickly; a behaviour *tree* is more powerful but more than you need here. You put this into practice in Task 2.

Task 1 – Build the Arm Control Stack

(25 pts)

The provided package `asmr_arm_description` spawns a two-link planar arm on a fixed pedestal in Gazebo — the same geometry you analysed in the Week 6 notebook. Your task is to build the arm control stack that drives it, and to prove it works by pushing a block.

The files you create are listed in Figure 2.

(a) **Kinematics server.** (5 pts)

Implement `kinematics_server` to advertise the provided services (`ComputeFK.srv`, `ComputeIK.srv`):

- `compute_fk` — given the two joint angles, returns the end-effector position and a flag reporting whether the request was valid.
- `compute_ik` — given a target end-effector position, returns a pair of joint angles that reach it and a flag that is false when the target lies outside the arm's workspace.

Link lengths & frames. Your FK/IK must use the link lengths from the relevant `arm_dimensions{...}.yaml` (the pedestal arm: `arm_dimensions_pedestal.yaml`, $l_1 = l_2 = 0.5\text{m}$), not the notebook's 1.0m. `tf2_echo` / `view_frames` are useful to inspect where the arm sits.

Grading: both services respond correctly to test calls, and the validity flag is set correctly for out-of-reach targets 5 pts

(b) **Tune the controller.** (5 pts)

Two provided files are *templates* you complete. In `arm.ros2_control.xacro`, declare each actuated joint's hardware interfaces — the command interface you drive and the state interfaces you read back (one joint is shown as an example). In `controllers_pedestal.yaml`, complete the `arm_pid_controller` block — its command/state interfaces and the per-joint gains; the comments list the valid options. With both filled, tune the gains until the arm, commanded to a joint configuration, *reaches and holds* it without drooping or sustained oscillation.

PID Tuning Approach. Before you have a trajectory server, drive the arm by hand: publish a single `control_msgs/MultiDOFCommand` setpoint to `/arm_pid_controller/reference` and watch `/joint_states` settle (rqt).

Grading: a commanded joint pose is reached and held steadily (no visible sag, no sustained oscillation) 5 pts

(c) **Trajectory server.** (10 pts)

Implement `trajectory_server` as the action server for the provided `ExecuteTrajectory.action`. Given a target end-effector pose, your server *designs and executes a smooth trajectory* to it — generating that motion is the heart of this task. The goal carries the target pose; the feedback reports progress; the result reports the outcome and the final configuration.

Your server must:

- find the joint angles for the target using the kinematics services
- *design a smooth trajectory* to it — interpolate a dense stream of intermediate setpoints from the current configuration toward the target so the arm sweeps there without jumping or lurching; how many steps, how fast, and how you pace them are yours to decide
- implement the full ROS 2 action contract: accept or reject the incoming goal, publish feedback while the goal runs, return a result on completion, and respond to a cancel request by stopping the arm
- stay responsive while a goal executes — it must still publish feedback and be able to accept a cancellation during motion.

Commanding the arm. The controller is commanded by publishing a `control_msgs/MultiDOFCommand` of target joint angles (radians) to `/arm_pid_controller/reference`. Live joint positions appear on `/joint_states` (`sensor_msgs/JointState`).

Staying responsive during motion. While the goal callback moves the arm, the server must still publish feedback and honour a cancel. On a single-threaded executor, a callback that waits (sleeping, or polling `/joint_states`) freezes the *whole* node — silently, with no error. A `MultiThreadedExecutor` with the callbacks in a `ReentrantCallbackGroup` lets them run concurrently — but now several threads touch the same state, so protect it with a `threading.Lock`.

Grading:

- Full action protocol — goal acceptance, live feedback, result, and cancellation all behave correctly 5 pts
- The end-effector reaches the target via a smooth, well-paced trajectory — no jump or lurch 5 pts

(d) **Push task.** (5 pts)

This is your first *mission protocol*: a fixed, pre-planned sequence of waypoints, run open-loop as the block is at a predefined position. Implement `push_block_mission` in `asmr_arm_mission`: a node that sends a single trajectory goal of **at least three named waypoints** — a home pose, an approach pose behind the block, and a push pose that displaces it. The mission node must use the action correctly: send the goal, confirm it was accepted, follow the feedback, and read the result.

Bringing up the scenario. Create the `asmr_arm_bringup` package and write `push_block_world.launch.py` in it — one launch that brings up the push-block world, the arm and its controllers, and your `kinematics_server` and `trajectory_server`. Launch the scenario, then fire the one-shot mission from a second terminal:

```
ros2 launch asmr_arm_bringup push_block_world.launch.py
ros2 run asmr_arm_mission push_block_mission
```

Grading: the stack comes up from your launch file, the mission node uses the action correctly (goal, feedback, result), and the block is visibly displaced 5 pts

Refresher pointers.

- Typed *interfaces* — Week 1, MiniLab 1.
- Service and action *clients* — Week 1, MiniLab 1.
- Forward and inverse kinematics of the two-link arm — Week 6 notebook.
- The arm's command and state topics — see Figure 3 and the provided `controllers_pedestal.yaml`.
- Executors and callback groups — *new this week*.
- Composing launch files (`IncludeLaunchDescription`) — Week 1, MiniLab 1.

Task 2 – Run the Mission: Mobile Manipulation in a Corridor

(25 pts)

Task 2 brings everything together. You mount the Task 1 arm onto your MiniLab 1 robot and send the combined system through the straight corridor. A hinged door blocks the far end. Your robot must drive the corridor, push the door open with its arm, and drive through into the room beyond.

You may build on *your own* MiniLab 1 robot or the provided reference robot — an updated version with 2 LiDARs mounted to the front of the robot. The provided and created files are listed in Figure 2; the spawn pose and door position are documented in the header of `corridor_world.sdf`.

(a) **Mount the arm.** (5 pts)

Add the arm macro to your robot's description and attach it to the base with a mounting transform that lets it push *ahead* of the robot. The macro brings its own `ros2_control` wiring; your job is the include and the mounting transform, worked out from the provided files and your Week 2 description.

Mobile-arm gains. For Task 2 you create `controllers_mobile.yaml`, modelled on the provided pedestal one. Your tuned Task 1 gains are a reasonable starting point, but the mobile arm is a different arm — expect to retune.

Grading: the combined URDF builds; the arm spawns on the robot 5 pts

(b) **Bring up the system.** (5 pts)

Write `my_robot_bringup/launch/corridor_world.launch.py`. A single command must start the whole stack:

```
ros2 launch my_robot_bringup corridor_world.launch.py
```

It must launch Gazebo with `corridor_world.sdf`, spawn your robot at the documented start pose ($x = -0.5$, $y = 0$, $z = 0.1$, yaw = 0.26 rad), start `robot_state_publisher` and the `ros_gz_bridge`, bring up the arm's controller, and start your Task 1 `kinematics_server` and `trajectory_server`. The `corridor_mission` node is *not* part of this launch — you start it separately (subtask c) once the stack is up.

Grading: the stack launches via the single command with declared arguments; all nodes start without errors 5 pts

(c) **Drive the corridor.** (5 pts)

In `my_robot_mission`, implement `corridor_mission`. Your node must realise two behaviours: a *home/fold* behaviour — fold the arm to a stowed pose before driving, so it does not foul the corridor — and a *drive* behaviour that advances along the corridor using LiDAR, staying centred, until the door is within reach. How you stay centred, how you determine when to stop, and how you encode the transitions and the conditions that fire them are your decisions. There is no prescribed algorithm.

Run the mission with:

```
ros2 run my_robot_mission corridor_mission
```

Driving the base. Drive the base by publishing `geometry_msgs/TwistStamped` on `/cmd_vel`; a provided velocity smoother limits acceleration and republishes to the simulator:

```
msg = TwistStamped()
msg.header.stamp = self.get_clock().now().to_msg()
msg.twist.linear.x = 0.2
msg.twist.angular.z = 0.0
```

Grading: the robot drives the full length of the corridor and stops reliably in front of the door 5 pts

(d) **Open the door and enter the room.** (10 pts)

Command the arm via your `ExecuteTrajectory` action client to push the door open, then drive the robot through the doorway and into the room. This subtask adds two more behaviours to your protocol: a *push* behaviour (turn a LiDAR reading into an arm target and open the door via the action) and a *pass* behaviour (advance into the room), ending the mission. How you translate a sensor reading into an arm target, how many push strokes you need, and when you decide the path is clear are your decisions. The `corridor_mission` node orchestrates both the navigation and the arm commands.

Grading:

- the arm reaches toward the door (a sensor reading is turned into an arm target on roughly the right line) 5 pts
- the door is visibly pushed open and the robot crosses the threshold into the room without collision 5 pts

Refresher pointers.

- Including a xacro file (`xacro:include`) and writing joints — Week 2
- The forward-facing scan index and the `sector_min` helper — Week 3 / MiniLab 1
- Sending an action goal and reading feedback/result — Week 1 `turtle_star`
- Launch files (with a `world` argument) — MiniLab 1